
Cold-Start Predictive Evaluation via Rasch-NCF Pipeline with Online Platt Calibration

Lisa St John

CS321M: AI Measurement Science

`lisa.stephan.sf@gmail.com`

Abstract

We describe our approach to the CS321M Predictive Evaluation Challenge. We adopt a three-stage pipeline: (1) a Rasch IRT model estimates subject ability θ and item difficulty β via Joint MLE; (2) a content-based Neural Collaborative Filtering (NCF) model maps text embeddings from a Sentence Transformer (`all-mpnet-base-v2`, 768-dim) together with a learned condition embedding (216 conditions $\times$ 32-dim) through an MLP head to produce logits for cold-start items; (3) online Platt temperature scaling calibrates probabilities using the small set of adaptively labeled examples each round. On 3-fold cold-start cross-validation, our method achieves Accuracy 0.800 ± 0.034 , AUC-ROC 0.900 ± 0.015 , with NLL coefficient of variation 3.2%. We also implement a farthest-point diversity acquisition function and analyze failure modes across benchmarks.

1 Problem Formulation and Method Description

1.1 Problem Definition

The Predictive Evaluation Challenge asks: given an AI system (subject) and a test question (item), predict the probability that the subject answers correctly—*without* running the evaluation. Formally, for subject s , item i , benchmark b , and test condition c :

$$\hat{p} = P(Y_{si} = 1 \mid \text{subject_content}, \text{item_content}, b, c) \quad (1)$$

All test items are cold-start (unseen during training), while subjects appear in the training set. The primary metric is negative log-loss (higher is better).

1.2 Three-Stage Pipeline

Stage 1: Rasch IRT Parameter Estimation. We use the Rasch model $P(Y = 1 \mid \theta, \beta) = \sigma(\theta - \beta)$ where θ is subject ability, β is item difficulty, and σ is the sigmoid function. Parameters are estimated via Joint Maximum Likelihood Estimation (JMLE) using Adam (lr=0.01) for 50 epochs with L_2 regularization ($\lambda = 10^{-4}$) and ReduceLROnPlateau scheduling. Training accuracy reaches 0.838.

Stage 2: Content-Based NCF. Since test items are cold-start, we learn a mapping from text content to prediction logits. A Sentence Transformer (`all-mpnet-base-v2`) encodes `subject_content` and `item_content` into L_2 -normalized 768-dim vectors. A condition embedding table (216 conditions $\times$ 32-dim, with an “unknown” fallback for unseen conditions) captures test-condition effects. The MLP head takes the concatenation $[u; v; c_{\text{emb}}] \in \mathbb{R}^{1568}$ through two hidden layers (256 units, BatchNorm, ReLU, Dropout) to produce a scalar logit.

Training uses BCEWithLogitsLoss, Adam (lr= 5×10^{-4} , weight decay 10^{-4}), gradient clipping (max norm 1.0), batch size 4096, up to 50 epochs with early stopping (patience=8). Model selection uses **validation NLL** (not accuracy) to align with the competition metric.

At inference, an LRU embedding cache (capacity 2000) avoids redundant Sentence Transformer calls—since the same subject repeats across all items in a round, caching reduces encoding calls by $\sim 50\%$ and prevents Codabench timeouts.

Stage 3: Online Platt Calibration. Each round, the platform reveals $K=5$ labeled examples per data category via adaptive labeling. We fit a temperature parameter T^* by grid search over $T \in [0.2, 5.0]$ (200 points), minimizing NLL on the labeled set: $p_{\text{cal}} = \sigma(\text{logit}/T^*)$. A safety guard ensures we only use $T^* \neq 1$ when calibration strictly improves NLL; otherwise we fall back to $T=1.0$. The calibrator auto-resets when the labeled set changes between rounds. Final probabilities are clipped to $[0.01, 0.99]$. On any exception, `predict()` falls back to the labeled base rate.

Acquisition Function. We implement greedy farthest-point diversity sampling: the first candidate seeds the pool, and subsequent candidates score by their minimum L_2 distance to all previously seen embeddings. This encourages label diversity without requiring pre-fit centroids. A `math.isfinite` guard prevents non-finite returns from triggering the platform’s random-selection fallback.

2 Training Data Description

We use the public `aims-foundations/measurement-db` dataset from HuggingFace. The raw dataset contains 10.7M response rows across 909 subjects and 103,983 items spanning multiple benchmarks.

Key preprocessing. Approximately 8.5% of responses are non-binary floats (from rating-scale benchmarks such as `ultrafeedback` and `mt_bench`). Since both the Rasch model and BCEWithLogitsLoss require binary labels, we *first* filter to response $\in \{0, 1\}$, *then* cast to int—this ordering prevents silent truncation (e.g., $0.7 \rightarrow 0$). The filtered dataset contains **4,448,512 rows**, **70,873 items**, and **216 test conditions**, with an overall pass rate of ~ 0.63 and response-matrix sparsity of $\sim 93\%$.

The data exhibits a long-tail distribution: a few large benchmarks (MMLU, GSM8K) contribute most rows, while many small benchmarks have few items. Item difficulty is bimodal—many items are very easy (>0.95 pass rate) or very hard (<0.05), with relatively few at moderate difficulty.

3 Experimental Results and Ablation Studies

3.1 Cold-Start Cross-Validation

We evaluate with item-stratified K -fold CV simulating the cold-start regime: test items never appear in training, while subjects are shared. Each fold uses 25 adaptive labels for Platt calibration.

Table 1: 3-fold cold-start cross-validation results.

Fold	Neg-NLL	Accuracy	AUC-ROC	vs. Baseline
1	−0.414	0.830	0.919	−33.9%
2	−0.393	0.820	0.897	−36.6%
3	−0.424	0.750	0.884	−36.2%
Mean±Std	−0.410 ± 0.013	0.800 ± 0.034	0.900 ± 0.015	−35.6%

The model consistently outperforms the base-rate baseline (predicting the global pass rate) across all folds, with NLL improvement averaging 35.6%. The NLL coefficient of variation is 3.2% ($<5\%$), indicating stable predictions across different item subsets.

3.2 Per-Benchmark Performance

`livecodebench` achieves the best performance (AUC 1.000), likely because coding tasks have clear difficulty gradients that the encoder captures well. `rewardbench` performs worst (AUC 0.766), likely because reward-model evaluation involves subtle semantic distinctions that a general-purpose encoder struggles with.

Table 2: Per-benchmark performance (mean $\pm$ std across 3 folds).

Benchmark	Neg-NLL	Accuracy	AUC-ROC
livecodebench	-0.179 ± 0.084	0.958 ± 0.059	1.000 ± 0.000
ai2d_test	-0.450 ± 0.120	0.791 ± 0.048	0.865 ± 0.040
bfcl	-0.496 ± 0.127	0.712 ± 0.052	0.866 ± 0.066
mathvista_mini	-0.551 ± 0.149	0.750 ± 0.083	0.738 ± 0.138
mmbench_v11	-0.293 ± 0.035	0.868 ± 0.079	0.684 ± 0.224
mlupro	-0.553 ± 0.188	0.729 ± 0.045	0.886 ± 0.064
rewardbench	-0.514 ± 0.098	0.741 ± 0.104	0.766 ± 0.104

3.3 Ablation Studies

Rasch baseline vs. NCF. The Rasch model (Stage 1) achieves training accuracy 0.838 but cannot handle cold-start items. NCF (Stage 2) lifts cold-start accuracy to 0.847 on the training validation split and 0.800 in cross-validation, confirming effective text-to-parameter generalization.

Condition embedding. The 216-condition embedding table lets the model distinguish the same item under different test conditions (e.g., zero-shot vs. chain-of-thought), capturing condition-specific performance variation.

Platt calibration. Online calibration with 25 labeled examples fits T^* via grid search. The safety fallback ($T=1$ when calibration hurts) ensures calibration never degrades performance, though improvement is modest with few labels.

Model selection by NLL. Selecting checkpoints by validation NLL (instead of accuracy) directly aligns with the competition metric, avoiding the risk of choosing high-accuracy but poorly-calibrated models.

Embedding cache. The LRU cache (capacity 2000) reduces total encoding calls by $\sim 50\%$ per round, critical for meeting the per-round time budget on Codabench.

Adaptive vs. random labeling. Farthest-point diversity acquisition shows only a 0.0006 NLL difference from random selection. Under the current single-temperature Platt framework, *which* examples are labeled matters less than having *some* labeled examples. Per-benchmark calibration or uncertainty-aware acquisition could improve this.

4 Failure Mode Analysis

Model stability. Early 5-fold CV showed NLL CV of 14.4%, indicating unstable predictions. After widening the temperature search range, adding auto-reset, and enabling embedding caching, NLL CV dropped to 3.2%. Residual variance comes from benchmark-level imbalance: large benchmarks provide strong training signal while small ones generalize poorly.

Non-binary responses. About 8.5% of raw responses are non-binary floats from rating-scale benchmarks (ultrafeedback, mt_bench). These caused loss explosion ($-2.5 \times 10^9 \rightarrow -7.5 \times 10^{12}$) before filtering. The filter-then-cast ordering is critical; reversed ordering silently truncates 0.7 to 0. These benchmarks are fully excluded post-filtering—if the test set includes their items, the model lacks targeted training signal.

Encoder limitations. all-mpnet-base-v2 is a general-purpose text encoder. For highly structured content (mathematical formulas, code snippets), encoding quality may degrade. The lower AUC on rewardbench (0.766) likely reflects this: reward-model evaluation requires fine-grained semantic discrimination beyond the encoder’s capacity.

Fundamental cold-start limits. Our method infers item difficulty from text alone, but difficulty often depends on subtle details invisible in the text. Two similar-looking math problems may differ drastically in difficulty due to a small numerical change—a fundamental limitation of pure content-based approaches.

Future directions. (1) 2PL/3PL IRT models with discrimination and guessing parameters; (2) cross-benchmark transfer to improve small-benchmark predictions; (3) uncertainty-aware acquisition combining predictive entropy with diversity; (4) domain-specific encoders for code and mathematics; (5) per-benchmark temperature calibration instead of a single global T ; (6) LayerNorm to eliminate BatchNorm’s reliance on running statistics at inference.